

UNIVERSIDAD DE EL SALVADOR

Facultad Multidisciplinaria De Occidente
Departamento de Ingeniería y Arquitectura

Ingeniería En Desarrollo De Software/Educación En Línea
LOGICA DE PROGRAMACION
SEMESTRE 1 / 2025



LABORATORIO NÚMERO 3

Arreglos de Estructuras, Arreglos como Miembros, Utilización de estructuras como parámetros y Uniones.

Nombre: _____, Carnet: _____
Fecha: _____, Grupo teórico: _____

Objetivo: Conocer los conceptos y el uso de los arreglos de estructuras, arreglos como miembros, estructuras como parámetros y uniones, además de exponer una breve introducción de estructuras en C++.

Indicaciones generales: Leer detenidamente cada una de las dos partes en que está dividida la actividad y brindar respuestas en las que se solicite.

Parte 1 - Breve introducción a las estructuras en C++,

Indicaciones: Leer detenidamente el contenido de la introducción a las estructuras, en el lenguaje de programación C++.

Las estructuras de datos permiten almacenar de manera ordenada una serie de valores dados en una misma variable. Las estructuras de datos más comunes son los arrays, que pueden ser unidimensionales (de una dimensión) también conocidos como vectores, o multidimensionales (de varias dimensiones) también conocidos como matrices.

Las estructuras de datos se emplean con el objetivo principal de organizar los datos contenidos dentro de la memoria del ordenador. Así, nuestra primera experiencia con estructuras comienza desde el momento mismo en que usamos en nuestros programas variables de tipos primitivos (char, short, int, float, etc), aunque hay otras un poco más

diferentes conocidas como **Estructuras compuestas**, como se verá en el siguiente apartado.

Estructuras compuestas (struct, union, class)

En C++ en orden de crear nuevas estructuras se emplean las palabras reservadas **struct**, **union** y **class**.

- **struct**: esta orden se emplea para agrupar variables de tipos iguales o diferentes en un solo registro, con la misma se pueden crear estructuras anónimas, estructuras con nombre, y un tipo especial de estructura conocida como bit-fields (banderas o campos de bits). Ejemplo de sintaxis para Struct:

```
struct [ <nombre tipo de estructura > ] {  
    [ <tipo> <nombre-variable[, nombre-variable, ...]> ] ;  
    [ <tipo> <nombre-variable[, nombre-variable, ...]> ] ;  
    ...  
} [ <variables de estructura> ] ;
```

- **union**: una union es similar a una struct, salvo que en una estructura creada con union los campos o variables comparten una dirección de almacenamiento común. La sintaxis para esta estructura es:

```
union [ <nombre tipo de union > ] {  
    <tipo> <lista de variables>;  
} [ <variables de union> ] ;
```

- **class**: una clase es una estructura en donde se agrupan variables y funciones, la misma es usada en Programación Orientada al Objeto. Las clases no son soportadas por el C estándar. Ejemplo del uso de class

```
class Pareja {  
    int a, b;  
public:  
    void setA(int n) { a = n; }  
    void setB(int n) { b = n; }  
    int getA() { return a; }  
    int getB() { return b; }  
    void mostrar() {  
        cout << "a = " << a << "; b = " << b << endl;  
    }  
} p1;
```

En este laboratorio ejemplificaremos de manera más detallada el uso de Struct, las otras estructuras complejas se abordarán según avance el contenido de la materia.

Ejemplo de uso de Struct:

Declarar en C++ una estructura para registrar los datos de tres libros como: título, autor y año.

```
1. #include <iostream>
2. using namespace std;
3. struct Libro{
4.     char titulo[20],
5.     autor[20];
6.     int anio;
7. };
```

En las líneas de código anterior se realizó la declaración de la estructura libro, para agregar y leer datos a la estructura Libro se puede hacer de la siguiente manera:

```
1. int main(){
2.     Libro listaLibro[2]; // declaramos una estructura de 3 libros
3.     for (int i = 0; i < 2; i++){
4.         cout << "\nEscriba el título del libro "<<i+1<<": ";
5.         cin >> listaLibro [i].titulo;
6.         cout << "\nEscriba el autor "<<i+1<<": ";
7.         cin >> listaLibro[i].autor;
8.         cout << "\nEscriba el año del libro "<<i+1<<": ";
9.         cin >> listaLibro [i].anio;
10.        cout << endl;
11.    }
12.    cout << "La lista de libros que se introdujeron es: \n\n";
13.
14.    for (int i = 0; i < 2; i++){
15.        cout << "\t" << listaLibro [i].titulo;
16.        cout << "\t" << listaLibro [i].autor;
17.        cout << "\t" << listaLibro[i].anio;
18.    }
19.    return 0;
20. }
```

Parte 2 – Realizar las siguientes actividades, ponderación de actividad 100%

Indicaciones: Desarrollar un programa, si se amerita, en el lenguaje de programación C++, que dé solución a cada punto que se muestra a continuación, tomando como referencia los tópicos abordados en clase.

1. Declarar una estructura en C++ que registre los datos de cinco personas como: Nombre, apellido, edad y teléfono (puede tomar como referencia el ejemplo de la parte 1).
2. Explique con sus palabras un uso en la vida real de un arreglo de estructuras, puede apoyarse de pseudocódigo o código en C++.
3. Defina con sus propias palabras que es una Union.
4. Mencione 2 diferencias entre una **union** y una **struct**.